



Agentic RAG Chatbot for Multi-Format Document QA using MCP

Coding Round Project Submission

Technical Implementation for AI/ML Systems Role

October 2025

N.Sushanth

Problem Statement & Requirements

Multi-Format Support

Build an intelligent RAG chatbot capable of processing and answering queries from uploaded documents across multiple file formats: PDF, DOCX, PPTX, CSV, and TXT files.

Agentic Architecture

Implement a distributed agent system using Model Context Protocol (MCP) for structured communication between specialized components handling document ingestion, retrieval, and response generation.

Conversational Intelligence

Deliver context-aware, conversational responses that maintain coherence across multi-turn conversations while providing source attribution for transparency and verification.



System Architecture Overview

Frontend Layer

Streamlit-based user interface providing intuitive file upload capabilities and real-time chat interaction. The interface maintains conversation history and displays source attributions for each response.

Backend Services

FastAPI-powered backend orchestrating three specialized agents:

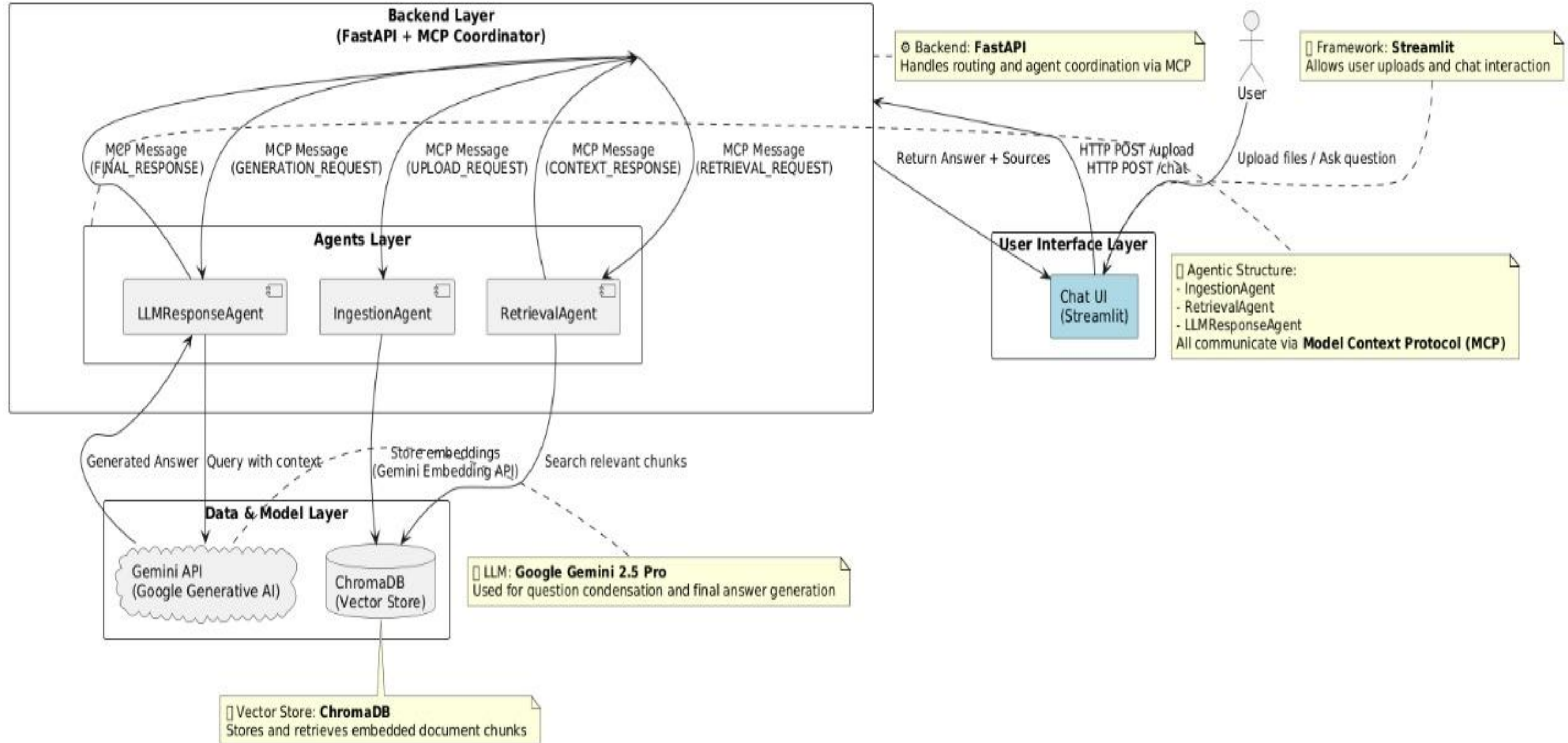
- **IngestionAgent** - Parses documents and generates embeddings
- **RetrievalAgent** - Performs semantic search with reranking
- **LLMResponseAgent** - Generates contextual responses using Gemini

Data Infrastructure

ChromaDB serves as the persistent vector store, while Google Gemini 2.5 Pro handles both embedding generation and response synthesis. MCP ensures structured, type-safe communication between all components.



Agentic RAG Chatbot Architecture (with MCP Messaging)



MCP Message Flow & Communication Protocol



Document Upload

User uploads documents through Streamlit UI, triggering MCP message to IngestionAgent for processing and embedding generation.



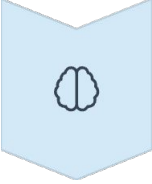
Vector Storage

Documents are parsed, chunked, and embedded using Gemini embedding model, then stored persistently in ChromaDB with metadata.



Query Processing

CoordinatorAgent receives user question and sends structured MCP message to RetrievalAgent for semantic search and context retrieval.



Response Generation

LLMResponseAgent receives context via MCP CONTEXT_RESPONSE message and generates final answer using Gemini with source attribution.



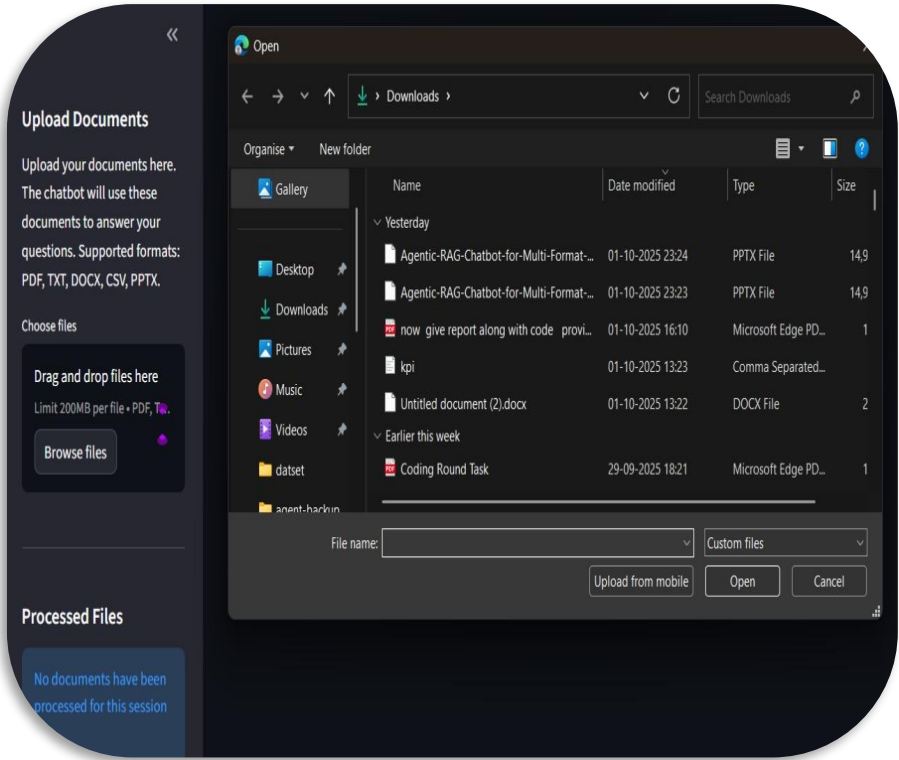
Example MCP Message Structure

```
{ "sender": "RetrievalAgent", "receiver": "LLMResponseAgent", "type": "CONTEXT_RESPONSE", "payload": { "query": "What KPIs were tracked?", "context": ["Slide 3: revenue growth...", "Doc: Q1 summary..."] }}
```

Technology Stack & Implementation Details

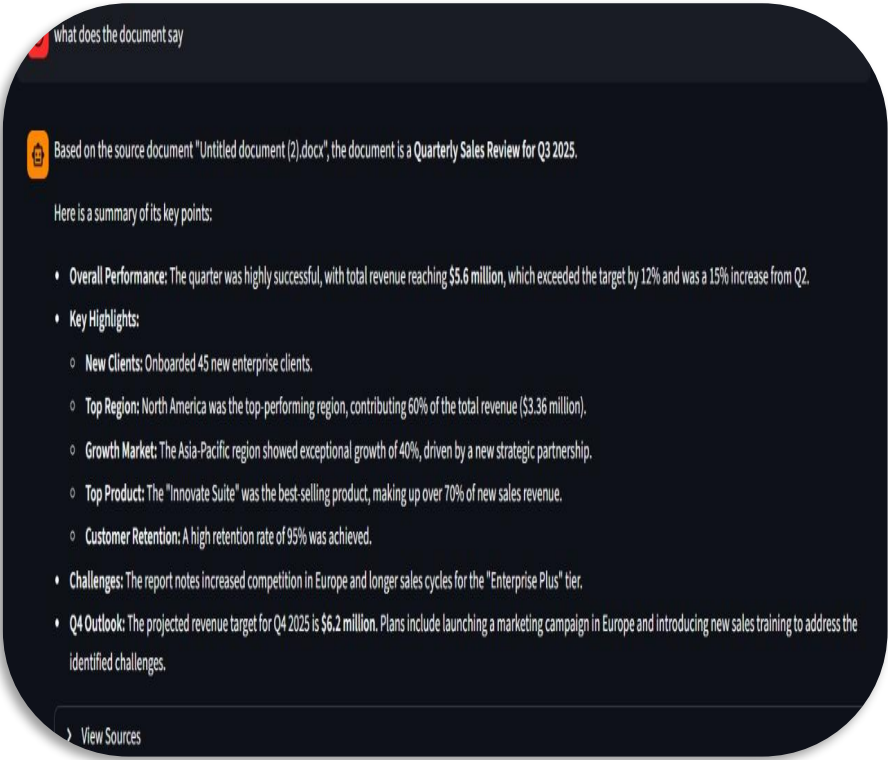
Layer	Technologies & Tools
Frontend	Streamlit for rapid UI development with file upload and chat components
Backend	FastAPI for high-performance async API endpoints and request handling
Agents	Python with LangChain for document processing, Pydantic for type validation
Vector DB	ChromaDB for persistent vector storage with metadata filtering capabilities
LLM	Google Gemini 2.5 Pro API for both embeddings and text generation
Protocol	Custom MCP implementation using Pydantic schemas for type-safe messaging

User Interface & Interaction Flow



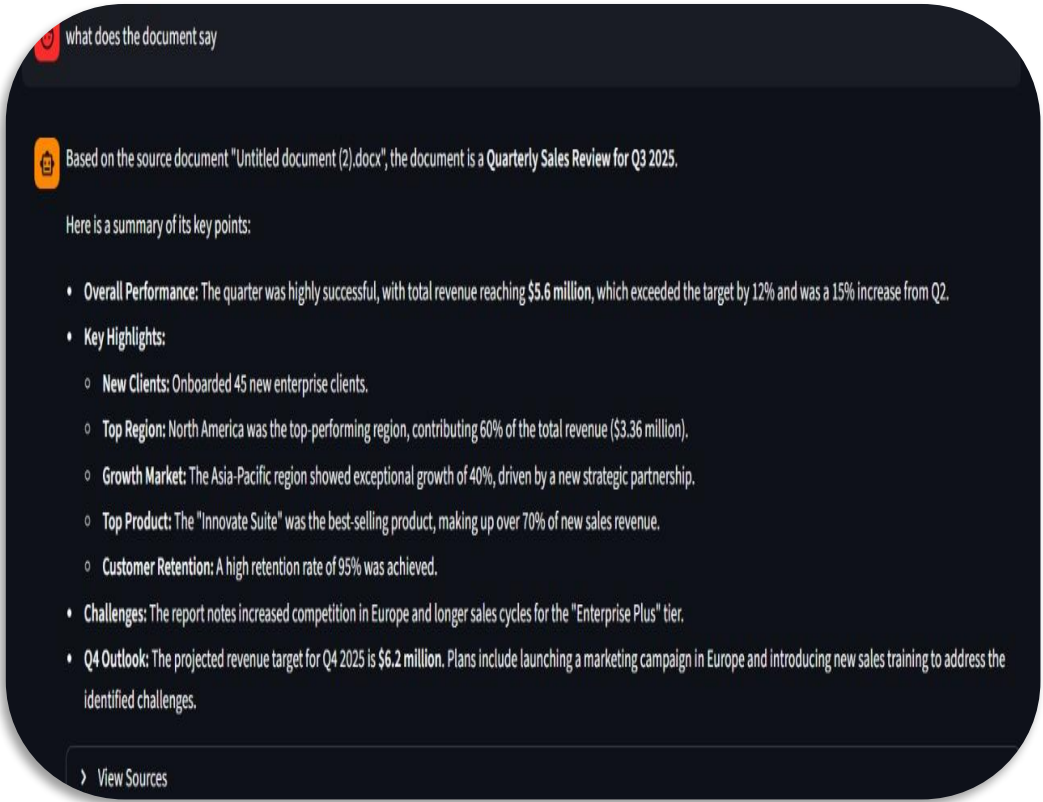
Document Upload Interface

Intuitive sidebar allowing users to upload multiple documents simultaneously. Progress indicators show processing status and successful ingestion confirmation.



Interactive Chat Experience

Real-time conversation interface displaying AI responses with clear source attributions. Users can ask follow-up questions while maintaining conversation context.



Contextual Response Display

Each response includes relevant document excerpts, source file names, and confidence indicators, enabling users to verify information accuracy and explore source materials.

Technical Challenges & Engineering Solutions

Lost in the Middle Problem

Challenge: The retrieval system worked for general queries but failed on metadata or references, as vector search prioritized body text while ignoring document edges.

Solution: I used multi-query expansion to capture diverse chunks and re-ranking to prioritize the ones directly answering the user's query.

Conversational Context

Challenge: The chatbot lacked memory, so it could not handle follow-up questions and lost the context of previous conversations.

Solution: I added a condense question step that rewrites follow-ups into complete standalone queries using the chat history, ensuring accurate retrieval.

Architecture & API Communication

Challenge: Separating frontend (Streamlit) and backend (FastAPI) introduced state and communication issues, like session mismatch and connection failures.

Solution: I used `st.session_state` to maintain session IDs and wrapped all API calls in error handling for reliable communication.

Dependency Conflicts

Challenge: Frequent version mismatches between libraries caused Import Errors and broke connections to Gemini models.

Solution: I enforced strict dependency control with `requirements.txt` and applied targeted upgrades alongside test scripts to isolate issues.

Future Enhancements & Project

Deliverables

Planned Improvements

- **Open-Source LLM Integration:** Replace Gemini with Mistral or Llama 3 for cost optimization and enhanced privacy
- **User Management System:** Implement authentication, user sessions, and document access controls
- **Performance Optimization:** Add Redis caching layer for faster retrieval and response times
- **Production Deployment:** Deploy on Hugging Face Spaces, Render, or AWS with auto-scaling capabilities

Additional Features

- Real-time collaboration and document sharing
- Advanced analytics and usage metrics
- Custom embedding fine-tuning for domain-specific knowledge

Complete Deliverables

- Comprehensive GitHub repository with full source code
- Detailed README with setup and deployment instructions
- Technical presentation (this document)
- Optional 5-minute demonstration video showcasing architecture, code walkthrough, and live application demo

